



Institut Teknologi Bandung

Direktorat Pendidikan Profesional Berkelanjutan

x

CHAPTER 4

Klasifikasi dan Regresi

Tiga alur kerja lengkap – biner, multikelas, dan regresi skalar – beserta aturan pilih loss dan aktivasi yang dipakai sisa buku ini.

Durasi 3 jam (2 sesi)

Pengajar Rahman Indra Kesuma, S.Kom., M.Cs.

Sumber Chollet & Watson, Deep Learning with Python 3e – bab 4

Tujuan Sesi

Setelah sesi ini, peserta dapat:

- 1 Memilih **aktivasi keluaran dan fungsi rugi** yang benar untuk klasifikasi biner, multikelas, dan regresi – tanpa menebak.
- 2 Menyiapkan data teks dengan **multi-hot encoding**, dan label dengan **one-hot** atau **sparse**.
- 3 Membaca **kurva rugi validasi** untuk menemukan titik berhenti terbaik, lalu melatih ulang sampai epoch itu saja.
- 4 Mengenali **leher botol informasi** dan menaksir ukuran lapis antara dari banyaknya kelas.
- 5 Menormalkan fitur dengan **statistik data latih**, dan menilai model berdata sedikit dengan **validasi silang K-lipat**.

BUKU

[Bab 4 — teks penuh](#)

NOTEBOOK

[01_imdb_klasifikasi_biner.ipynb](#)

NOTEBOOK

[02_reuters_multikelas.ipynb](#)

NOTEBOOK

[03_regresi_harga_rumah.ipynb](#)

REPO

[Notebook resmi penulis](#)

Dua belas istilah yang dipakai sampai bab 20

ISTILAH	ARTINYA	JENIS TUGAS	CIRINYA
Sample / input	Satu titik data yang masuk ke model.	Binary classification	Dua kategori, saling meniadakan.
Prediction / output	Yang keluar dari model.	Multiclass	Lebih dari dua kategori, satu label per sampel.
Target	Jawaban benar, dari sumber di luar model.	Multilabel	Satu sampel boleh punya beberapa label.
Loss value	Ukuran jarak prediksi ke target.	Scalar regression	Satu nilai kontinu.
Classes	Himpunan label yang mungkin.	Vector regression	Beberapa nilai kontinu sekaligus.
Mini-batch	Lazimnya 8-128 sampel sekaligus.		

Membedakan **multiclass** dari **multilabel** menentukan pilihan aktivasi keluaran: **softmax** untuk yang pertama (jumlahnya 1), **sigmoid per kelas** untuk yang kedua.

01

Klasifikasi biner: ulasan IMDB

50.000 ulasan film, positif atau negatif.

Data, dan cara mengubah deret kata jadi tensor

Listing 4.1-4.3 – muat dan vektorkan

PYTHON

```
import numpy as np
from keras.datasets import imdb

(train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words=10000)

def multi_hot_encode(sequences, num_classes):
    results = np.zeros((len(sequences), num_classes))
    for i, sequence in enumerate(sequences):
        results[i][sequence] = 1.0 # indeks kata yang muncul -> 1
    return results

x_train = multi_hot_encode(train_data, num_classes=10000)
x_test = multi_hot_encode(test_data, num_classes=10000)
y_train = train_labels.astype("float32")
y_test = test_labels.astype("float32")

print(x_train.shape, x_train[0][:12])
```

OUTPUT

```
(25000, 10000) [0. 1. 1. 0. 1. 1. 1. 1. 1. 1. 0. 0.]
```

- 25.000 latih + 25.000 uji, seimbang 50:50 positif-negatif.
- `num_words=10000` hanya menyimpan **10.000 kata tersering**; sisanya dibuang.
- Multi-hot **membuang urutan kata**. Cukup untuk sentimen; bab 14-15 menggantinya saat urutan mulai penting.

Model, validasi, dan titik balik di epoch 4

listing 4.4-4.6

PYTHON

```
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

x_val, partial_x = x_train[:10000], x_train[10000:]
y_val, partial_y = y_train[:10000], y_train[10000:]

history = model.fit(
    partial_x, partial_y,
    epochs=20, batch_size=512,
    validation_data=(x_val, y_val))
```



Rugi latih turun terus; rugi validasi berbalik naik setelah epoch 4.

Setelah epoch keempat, Anda mengoptimalkan berlebihan pada data latih, dan berakhir mempelajari representasi yang khas untuk data latih itu saja – yang tidak berlaku di luarnya.

Chollet & Watson, bab 4

Hasilnya, dan tolok banding yang harus dikalahkan

88%

akurasi uji setelah dilatih ulang 4 epoch

50%

tolok banding acak (kelasnya seimbang)

16

unit per lapis antara – model sengaja kecil

Tanpa fungsi aktivasi seperti `relu` (disebut juga nonlinearitas), lapis `Dense` hanya terdiri dari dua operasi linear – hasil kali titik dan penjumlahan. Ruang hipotesis seperti itu terlalu sempit.

Chollet & Watson, bab 4

Crossentropy datang dari teori informasi: ia mengukur **jarak antara dua sebaran peluang**. Itulah sebabnya ia pasan-gan alami bagi keluaran sigmoid dan softmax, dan bukan MSE.

02

Multikelas: kawat berita Reuters

46 topik, saling meniadakan.

One-hot atau sparse – antarmuka beda, hitungannya sama

label one-hot

PYTHON

```
from keras.utils import to_categorical

y_train = to_categorical(train_labels)
y_test = to_categorical(test_labels)
# bentuknya (8982, 46)

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"])
```

label sparse (bilangan bulat)

PYTHON

```
y_train = train_labels # biarkan bulat
y_test = test_labels
# bentuknya (8982,)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

Listing 4.11 – model dan metrik top-K

PYTHON

```
model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(46, activation="softmax"), # satu unit per kelas
])
top_3 = keras.metrics.TopKCategoricalAccuracy(k=3, name="top_3_accuracy")
model.compile(optimizer="adam", loss="categorical_crossentropy",
    metrics=["accuracy", top_3])
```

Top-K accuracy menanyakan hal yang berbeda: apakah kelas yang benar ada di antara k tebakan teratas? Untuk 46 kelas, itu ukuran yang **jauh lebih berguna** bagi sistem yang hanya menyarankan.

Leher botol informasi: percobaan yang sengaja digagalkan



Lapis 4 unit di tengah memaksa 46 kelas diperas ke 4 dimensi. Turun 8 poin akurasi.

Model itu sanggup menjejalkan sebagian besar informasi yang diperlukan ke dalam representasi 4 dimensi – tetapi tidak semuanya.

Chollet & Watson, bab 4

Aturan praktisnya: **lapis antara tidak boleh lebih sempit dari jumlah kelas keluaran**. Itulah sebabnya contoh IMDB cukup 16 unit (2 kelas), sementara Reuters butuh 64.

Melatih ulang di epoch terbaik

listing 4.14 – model produksi

PYTHON

```
model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(46, activation="softmax"),
])
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
model.fit(x_train, y_train, epochs=9, batch_size=512) # g = titik terbaik tadi

results = model.evaluate(x_test, y_test)
print(results)
```

OUTPUT

```
71/71 ---- 0s 3ms/step - accuracy: 0.7969 - loss: 0.9127
[0.9127, 0.7969]
```

~80%

akurasi uji

~19%

tolok banding acak – 46 kelas, sebaran tak rata

Selalu hitung **tolok banding akal sehat** dulu. Angka 80% terdengar biasa saja sampai Anda tahu bahwa menebak asal hanya memberi 19%.

03

Regresi skalar: harga rumah

Data sedikit. Aturan mainnya berubah.

Normalisasi fitur – dan jebakan kebocoran data

listing 4.16-4.17 – muat dan normalkan

PYTHON

```
from keras.datasets import california_housing

(train_data, train_targets), (test_data, test_targets) = (
    california_housing.load_data(version="small"))
# 480 latih, 120 uji, 8 fitur numerik per distrik

mean = train_data.mean(axis=0)
std = train_data.std(axis=0)
x_train = (train_data - mean) / std
x_test = (test_data - mean) / std    # PAKAI statistik data LATIH, bukan uji

y_train = train_targets / 100000    # skalakan target ke rentang yang wajar
y_test = test_targets / 100000
```

Baris `x_test = (test_data - mean) / std` memakai `mean` dan `std` dari data **latih**. Menghitung ulang dari data uji adalah **kebocoran informasi**: model jadi tahu sesuatu tentang data yang seharusnya belum pernah dilihatnya. Bab 5 menamainya secara resmi.

- 8 fitur: bujur, lintang, umur rumah, populasi, jumlah rumah tangga, penghasilan median, total kamar, total kamar tidur.
- Target: harga rumah median, kontinu, kira-kira \$60 ribu sampai \$500 ribu.
- Rentang tiap fitur berbeda jauh; tanpa normalisasi **optimalisasi jadi kacau**.

Model regresi: keluaran tanpa aktivasi

listing 4.18

PYTHON

```
def get_model():  
    model = keras.Sequential([  
        layers.Dense(64, activation="relu"),  
        layers.Dense(64, activation="relu"),  
        layers.Dense(1),          # TANPA aktivasi - bebas menebak nilai apa pun  
    ])  
    model.compile(optimizer="adam",  
                  loss="mean_squared_error",  
                  metrics=["mean_absolute_error"])  
    return model
```

Tanpa aktivasi keluaran

Sigmoid akan mengurung tebakan ke $[0,1]$. Regresi harus bebas.

Model sengaja kecil

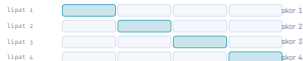
480 sampel saja. Model besar akan langsung menghafalnya.

MSE untuk rugi, MAE untuk metrik

MAE bisa dibaca manusia: MAE 0,5 $\approx$ meleset \$50 ribu.

Validasi silang K-liput, karena datanya sedikit

data latih dibagi 4; tiap lipatan sekali jadi validasi



skor akhir = rerata keempatnya

Dengan 480 sampel, satu belahan validasi tunggal terlalu besik – skornya bergantung pada belahan mana yang kebetulan terpilih.

Listing 4.19 – Inti gelung K-liput

PYTHON

```
k, num_epochs, all_scores = 4, 50, []
num_val_samples = len(x_train) // k

for i in range(k):
    fold_x_val = x_train[i * num_val_samples : (i + 1) * num_val_samples]
    fold_y_val = y_train[i * num_val_samples : (i + 1) * num_val_samples]
    fold_x_train = np.concatenate(
        [x_train[i * num_val_samples], x_train[(i + 1) * num_val_samples :]], axis=0)
    fold_y_train = np.concatenate(
        [y_train[i * num_val_samples], y_train[(i + 1) * num_val_samples :]], axis=0)

    model = get_model()
    model.fit(fold_x_train, fold_y_train, epochs=num_epochs, batch_size=16, verbose=0)
    val_loss, val_mae = model.evaluate(fold_x_val, fold_y_val, verbose=0)
    all_scores.append(val_mae)
```

OUTPUT

```
[0.265, 0.292, 0.232, 0.349]
rerata MAE: 0.296 -> meleset sekitar $29.600
```

Berapa lama melatihnya? Tanya kurvanya

listing 4.20 – rerata kurva MAE

PYTHON

```
all_mae_histories = []
for i in range(k):
    # ... penyiapan lipatan sama seperti sebelumnya ...
    history = model.fit(fold_x_train, fold_y_train,
                        validation_data=(fold_x_val, fold_y_val),
                        epochs=200, batch_size=16, verbose=0)
    all_mae_histories.append(history.history["val_mean_absolute_error"])

average_mae_history = [
    np.mean([h[i] for h in all_mae_histories]) for i in range(200)
]
```

MAE validasi **mendatar di sekitar epoch 120-140**, lalu mulai memburuk. Itulah titik berhentinya.

~0,31

MAE uji akhir – meleset sekitar \$31.000

model akhir

PYTHON

2,83

Tabel yang akan Anda pakai terus

JENIS TUGAS	AKTIVASI LAPIS AKHIR	FUNGSI RUGI	METRIK LAZIM
Klasifikasi biner	<code>sigmoid</code> (1 unit)	<code>binary_crossentropy</code>	accuracy
Multikelas , label one-hot	<code>softmax</code> (N unit)	<code>categorical_crossentropy</code>	accuracy, top-K
Multikelas , label bulat	<code>softmax</code> (N unit)	<code>sparse_categorical_crossentropy</code>	accuracy
Multilabel	<code>sigmoid</code> (N unit)	<code>binary_crossentropy</code>	accuracy, AUC
Regresi skalar	tanpa aktivasi (1 unit)	<code>mean_squared_error</code>	MAE

- 1 Vektorkan data diskret; **normalkan fitur dengan statistik data latih**.
- 2 Lapis antara **tidak boleh lebih sempit dari jumlah kelas** – leher botol.
- 3 Data sedikit → model kecil, satu atau dua lapis antara.
- 4 Latih sampai overfit untuk **melihat** titik baliknya, lalu latih ulang sampai titik itu.
- 5 Data sedikit → **K-lipat**, bukan satu belahan validasi.

NOTEBOOK

[01_imdb_klasifikasi_biner.ipynb](#)

BAB BERIKUT

[Bab 5 — Dasar machine learning](#)